

Лабораторная работа № 12

Программирование в командном процессоре ОС UNIX

Пириев Керем

Содержание

1	Цель работы	5
2	Задание	6
3	Выполнение лабораторной работы	7
3.1	1. Создание рабочего каталога	7
3.2	2. Скрипт 1: резервное копирование самого себя	7
3.3	3. Скрипт 2: обработка произвольного числа аргументов	8
3.4	4. Скрипт 3: аналог команды ls (без использования ls)	8
3.5	5. Скрипт 4: подсчёт файлов по расширению	9
4	Вывод	11
5	Контрольные вопросы	12
5.1	1. Объясните понятие командной оболочки. Приведите примеры командных оболочек. Чем они отличаются?	12
5.2	2. Что такое POSIX?	12
5.3	3. Как определяются переменные и массивы в языке программирования bash?	12
5.4	4. Каково назначение операторов let и read?	13
5.5	5. Какие арифметические операции можно применять в языке программирования bash?	13
5.6	6. Что означает операция (())?	13
5.7	7. Какие стандартные имена переменных Вам известны?	13
5.8	8. Что такое метасимволы?	13
5.9	9. Как экранировать метасимволы?	13
5.10	10. Как создавать и запускать командные файлы?	14
5.11	11. Как определяются функции в языке программирования bash?	14
5.12	12. Каким образом можно выяснить, является файл каталогом или обычным файлом?	14
5.13	13. Каково назначение команд set, typeset и unset?	14
5.14	14. Как передаются параметры в командные файлы?	14
5.15	15. Назовите специальные переменные языка bash и их назначение.	15
	Список литературы	16

Список иллюстраций

3.1	Создание рабочего каталога	7
3.2	Создание скрипта backup_self.sh в nano	7
3.3	Запуск backup_self.sh и проверка архива	8
3.4	Редактирование скрипта show_args.sh	8
3.5	Запуск show_args.sh с аргументами	8
3.6	Редактирование скрипта my_ls.sh	9
3.7	Запуск my_ls.sh для текущего каталога и /etc	9
3.8	Редактирование скрипта count_ext.sh	9
3.9	Запуск count_ext.sh и результат подсчета	10

Список таблиц

1 Цель работы

Изучить основы программирования в оболочке bash: переменные, циклы, условия, работа с аргументами, тестирование файлов.

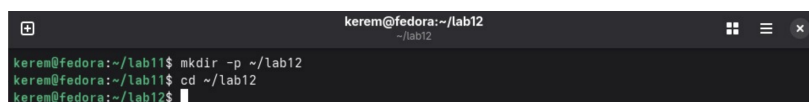
2 Задание

1. Создать рабочий каталог `~/lab12` для размещения всех скриптов.
2. Создать скрипт резервного копирования самого себя (`backup_self.sh`).
3. Создать скрипт обработки произвольного числа аргументов (`show_args.sh`).
4. Создать скрипт-эмулятор команды `ls` (`my_ls.sh`).
5. Создать скрипт подсчёта файлов по расширению (`count_ext.sh`).
6. Дать ответы на контрольные вопросы.

3 Выполнение лабораторной работы

3.1 1. Создание рабочего каталога

Создан каталог `~/lab12` для размещения всех скриптов (рис. Рисунок 3.1).

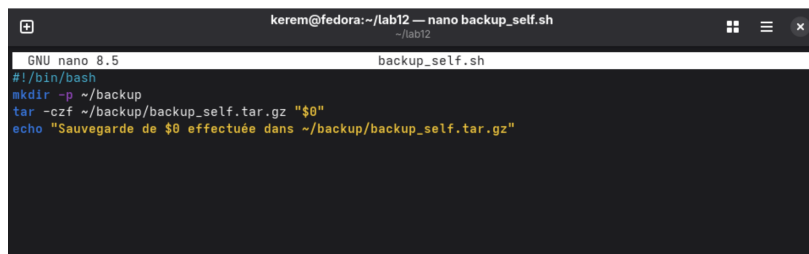


```
kerem@fedora:~/lab12$ mkdir -p ~/lab12
kerem@fedora:~/lab12$ cd ~/lab12
kerem@fedora:~/lab12$
```

Рисунок 3.1: Создание рабочего каталога

3.2 2. Скрипт 1: резервное копирование самого себя

Создан файл `backup_self.sh` с помощью редактора nano (рис. Рисунок 3.2).



```
GNU nano 8.5 backup_self.sh
#!/bin/bash
mkdir -p ~/backup
tar -czf ~/backup/backup_self.tar.gz "$0"
echo "Sauvegarde de $0 effectuée dans ~/backup/backup_self.tar.gz"
```

Рисунок 3.2: Создание скрипта `backup_self.sh` в nano

Выполнен запуск скрипта и проверка содержимого резервной копии (рис. Рисунок 3.3).

```

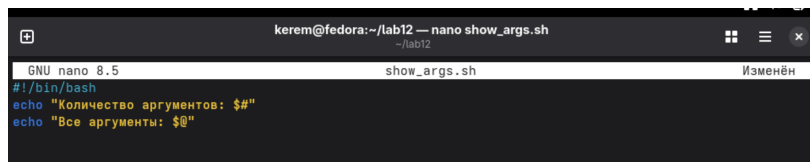
kerem@fedora:~/lab12$ nano backup_self.sh
kerem@fedora:~/lab12$ chmod +x backup_self.sh
./backup_self.sh
Sauvegarde de ./backup_self.sh effectuée dans ~/backup/backup_self.tar.gz
kerem@fedora:~/lab12$ ls -l ~/backup/
итого 12
-rw-r--r--. 1 kerem kerem 220 мая 31 03:53 backup_self.tar.gz
-rw-r--r--. 1 kerem kerem 396 мая 4 03:53 backup.sh_20260504_035312.tar.bz2
-rw-r--r--. 1 kerem kerem 396 мая 4 04:10 backup.sh_20260504_041048.tar.bz2

```

Рисунок 3.3: Запуск backup_self.sh и проверка архива

3.3 3. Скрипт 2: обработка произвольного числа аргументов

Создан файл show_args.sh (рис. Рисунок 3.4).



```

kerem@fedora:~/lab12 — nano show_args.sh
GNU nano 8.5 show_args.sh Изменён
#!/bin/bash
echo "Количество аргументов: $#"
```

Рисунок 3.4: Редактирование скрипта show_args.sh

Выполнен запуск скрипта с 12 аргументами (рис. Рисунок 3.5).

```

kerem@fedora:~/lab12$ chmod +x show_args.sh
kerem@fedora:~/lab12$ ./show_args.sh a b c d e f g h i j k l
Количество аргументов: 12
Все аргументы: a b c d e f g h i j k l
kerem@fedora:~/lab12$

```

Рисунок 3.5: Запуск show_args.sh с аргументами

3.4 4. Скрипт 3: аналог команды ls (без использования ls)

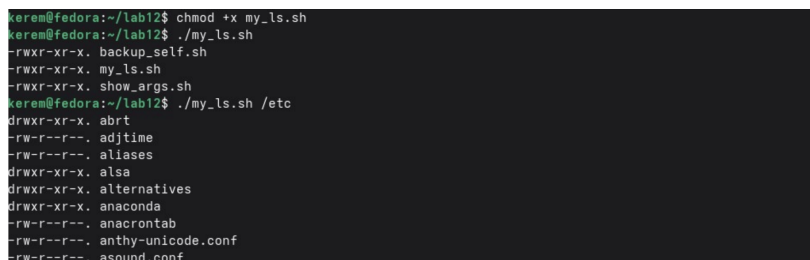
Создан файл my_ls.sh, эмулирующий вывод команды ls -l (рис. Рисунок 3.6).



```
kerem@fedora:~/lab12 — nano my_ls.sh
GNU nano 8.5 my_ls.sh
#!/bin/bash
dir="${1:-.}"
for file in "$dir"/*; do
  if [ -e "$file" ]; then
    permissions=$(ls -ld "$file" | cut -d' ' -f1)
    echo "$permissions $(basename "$file")"
  fi
done
```

Рисунок 3.6: Редактирование скрипта my_ls.sh

Выполнен запуск скрипта для текущего каталога и каталога /etc (рис. Рисунок 3.7).

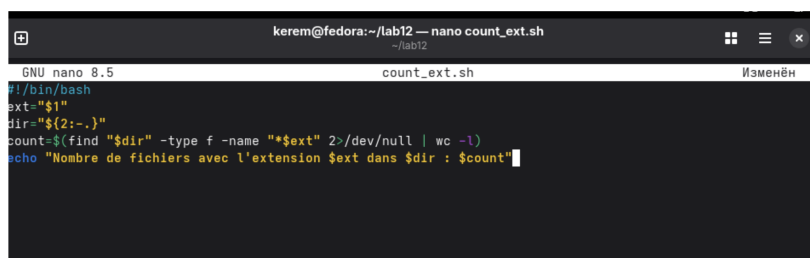


```
kerem@fedora:~/lab12$ chmod +x my_ls.sh
kerem@fedora:~/lab12$ ./my_ls.sh
-rwxr-xr-x. backup_self.sh
-rwxr-xr-x. my_ls.sh
-rwxr-xr-x. show_args.sh
kerem@fedora:~/lab12$ ./my_ls.sh /etc
drwxr-xr-x. abrt
-rw-r--r--. adjtime
-rw-r--r--. aliases
drwxr-xr-x. alsa
drwxr-xr-x. alternatives
drwxr-xr-x. anaconda
-rw-r--r--. anacrontab
-rw-r--r--. anthy-unicode.conf
-rw-r--r--. asound.conf
```

Рисунок 3.7: Запуск my_ls.sh для текущего каталога и /etc

3.5 5. Скрипт 4: подсчёт файлов по расширению

Создан файл count_ext.sh для подсчета файлов с заданным расширением (рис. Рисунок 3.8).



```
kerem@fedora:~/lab12 — nano count_ext.sh
GNU nano 8.5 count_ext.sh
#!/bin/bash
ext="${1}"
dir="${2:-.}"
count=$(find "$dir" -type f -name "*$ext" 2>/dev/null | wc -l)
echo "Nombre de fichiers avec l'extension $ext dans $dir : $count"
```

Рисунок 3.8: Редактирование скрипта count_ext.sh

Выполнен запуск скрипта после исправления для поиска файлов .txt в /etc (рис. Рисунок 3.9).

```
kerem@fedora:~/lab12$ nano count_ext.sh
kerem@fedora:~/lab12$ chmod +x count_ext.sh
kerem@fedora:~/lab12$ ./count_ext.sh .txt /etc
Nombre de fichiers avec l'extension .txt dans /etc : 20
kerem@fedora:~/lab12$
```

Рисунок 3.9: Запуск count_ext.sh и результат подсчета

4 Вывод

В ходе работы были написаны и отлажены четыре bash-скрипта: 1. Резервное копирование самого себя — скрипт создаёт архив с собственным исходным кодом. 2. Обработка произвольного количества аргументов — скрипт выводит число и список всех переданных аргументов. 3. Эмуляция команды `ls` — скрипт выводит содержимое каталога с правами доступа без использования встроенной команды `ls`. 4. Подсчёт файлов заданного расширения — скрипт подсчитывает количество файлов с указанным расширением в заданном каталоге. Получены практические навыки работы с переменными, циклами, условиями, передачей параметров и тестированием файлов в оболочке bash.

5 Контрольные вопросы

5.1 1. Объясните понятие командной оболочки.

Приведите примеры командных оболочек. Чем они отличаются?

Командная оболочка (shell) — это программа, обеспечивающая интерфейс между пользователем и операционной системой, интерпретирующая команды, вводимые пользователем. Примеры: sh, bash, csh, zsh, fish. Отличаются синтаксисом и дополнительными возможностями.

5.2 2. Что такое POSIX?

POSIX (Portable Operating System Interface) — набор стандартов, описывающих интерфейсы между ОС и прикладными программами для обеспечения переносимости.

5.3 3. Как определяются переменные и массивы в языке программирования bash?

Переменная: `var=value`. Массив: `arr=(val1 val2 val3)`.

5.4 4. Каково назначение операторов `let` и `read`?

`let` — для арифметических операций. `read` — для чтения ввода пользователя.

5.5 5. Какие арифметические операции можно применять в языке программирования `bash`?

`+`, `-`, `*`, `/`, `%`, `+=`, `-=`, `*=`, `/=`, `++`, `--`.

5.6 6. Что означает операция `(( ))`?

Конструкция для выполнения арифметических операций в стиле языка C без использования `$` перед переменными.

5.7 7. Какие стандартные имена переменных Вам известны?

`HOME`, `PATH`, `USER`, `SHELL`, `PWD`, `PS1`, `IFS`, `LOGNAME`.

5.8 8. Что такое метасимволы?

Символы, имеющие специальное значение для оболочки (`*`, `?`, `[ ]`, `&`, `;` и др.).

5.9 9. Как экранировать метасимволы?

С помощью обратного слеша (`\`), одинарных (`' * '`) или двойных (`" * "`) кавычек.

5.10 10. Как создавать и запускать командные файлы?

Создать файл с `#!/bin/bash`, сделать исполняемым (`chmod +x`), запустить через `./file`.

5.11 11. Как определяются функции в языке программирования bash?

Через ключевое слово `function name { ... }` или `name() { ... }`.

5.12 12. Каким образом можно выяснить, является файл каталогом или обычным файлом?

С помощью условных операторов `test` или `[ ]` с ключами `-d` (каталог) и `-f` (файл).

5.13 13. Каково назначение команд `set`, `typeset` и `unset`?

`set` — параметры оболочки и переменные; `typeset` — объявление переменных с атрибутами; `unset` — удаление переменных или функций.

5.14 14. Как передаются параметры в командные файлы?

Через пробел после имени скрипта при запуске (`./script.sh arg1 arg2`).

5.15 15. Назовите специальные переменные языка **bash** и их назначение.

- \$# — количество аргументов.
- \$* — все аргументы как одна строка.
- @\$ — все аргументы как отдельные строки.
- \$? — код возврата последней команды.
- \$\$ — PID текущей оболочки.
- \$! — PID последнего фонового процесса.
- \$0 — имя скрипта.

Список литературы

1. *Newham C.* Learning the bash Shell: Unix Shell Programming. — O'Reilly Media, 2005. — 354 p. — (In a Nutshell). — ISBN 0596009658. — URL: <http://www.amazon.com/Learning-bash-Shell-Programming-Nutshell/dp/0596009658>.
2. *Robbins A.* Bash Pocket Reference. — O'Reilly Media, 2016. — 156 p. — ISBN 978-1491941591.
3. *Zarrelli G.* Mastering Bash. — Packt Publishing, 2017. — 502 p. — ISBN 9781784396879.
4. *Таненбаум Э., Бос Х.* Современные операционные системы.. — 4-е изд. — СПб. : Питер, 2015.. — 1120 с. — (Классика Computer Science).